

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
```

```
print("Files in colab:")
print(os.listdir("/kaggle"))
```

```
Files in colab:
['bm_sales.csv', 'bm_inventory.csv', 'input', 'bm_stores.csv', 'bm_customers.csv', 'bm_promotions.csv', 'bm_skus.csv']
```

```
customers = pd.read_csv("/kaggle/bm_customers.csv")
inventory = pd.read_csv("/kaggle/bm_inventory.csv")
promotions = pd.read_csv("/kaggle/bm_promotions.csv")
sales = pd.read_csv("/kaggle/bm_sales.csv")
skus = pd.read_csv("/kaggle/bm_skus.csv")
stores = pd.read_csv("/kaggle/bm_stores.csv")

print("customers: ", customers.shape)
print("inventory: ", inventory.shape)
print("promotions: ", promotions.shape)
print("sales: ", sales.shape)
print("skus: ", skus.shape)
print("stores: ", stores.shape)
```

```
customers: (5000, 7)
inventory: (8735, 7)
promotions: (33, 6)
sales: (641843, 9)
skus: (200, 7)
stores: (50, 5)
```

```
datasets={
    "customers" : customers,
    "inventory" : inventory,
    "promotions" : promotions,
    "sales" : sales,
    "skus" : skus,
    "stores" : stores
}
for name,df in datasets.items():
    print("\n" + '=' * 60)
    print(name)
    print("="*60)

    print("shape:", df.shape)
    print("\ncolumns:")
    print(df.columns.tolist())

    print("\n missing value:")
    print(df.isnull().sum())

print("\n Duplicate rows:",df.duplicated().sum())
```

```

shape: (200, 7)

columns:
['sku_id', 'sku_name', 'category', 'subcategory', 'unit_price', 'cost_price', 'brand']

missing value:
sku_id      0
sku_name    0
category    0
subcategory 0
unit_price  0
cost_price  0
brand       0
dtype: int64

=====
stores
=====
shape: (50, 5)

columns:
['store_id', 'store_name', 'city', 'store_type', 'opening_date']

missing value:
store_id      0
store_name    0
city          0
store_type    0
opening_date  0
dtype: int64

Duplicate rows: 0

```

```
# Check numeric columns for negative values
```

```

print("Negative quantities:", (sales["quantity"] < 0).sum())
print("Negative unit prices:", (sales["unit_price"] < 0).sum())
print("Negative total values:", (sales["total_value"] < 0).sum())
print("Negative discounts:", (sales["discount_pct"] < 0).sum())

print("\nInventory:")
print("Negative stock:", (inventory["stock_on_hand"] < 0).sum())
print("Negative reorder point:", (inventory["reorder_point"] < 0).sum())
print("Negative safety stock:", (inventory["safety_stock"] < 0).sum())

```

```

Negative quantities: 0
Negative unit prices: 0
Negative total values: 0
Negative discounts: 0

Inventory:
Negative stock: 0
Negative reorder point: 0
Negative safety stock: 0

```

```

print("Sales date range:")
print("Start:", sales["date"].min())
print("End:", sales["date"].max())

print("\nPromotion date range:")
print("Start:", promotions["start_date"].min())
print("End:", promotions["end_date"].max())

print("\nInventory snapshot date range:")
print("Start:", inventory["snapshot_date"].min())
print("End:", inventory["snapshot_date"].max())

```

```

Sales date range:
Start: 2021-01-01
End: 2025-10-31

Promotion date range:
Start: 2021-01-01
End: 2025-08-31

Inventory snapshot date range:
Start: 2025-10-31
End: 2025-10-31

```

```

print("Unique SKUs in Sales:", sales["sku_id"].nunique())
print("Unique SKUs in SKU Master:", skus["sku_id"].nunique())

print("\nUnique Stores in Sales:", sales["store_id"].nunique())
print("Unique Stores in Store Master:", stores["store_id"].nunique())

```

```
print("\nUnique Customers in Sales:", sales["customer_id"].nunique())
print("Unique Customers in Customer Master:", customers["cust_id"].nunique())
```

Unique SKUs in Sales: 200
Unique SKUs in SKU Master: 200

Unique Stores in Sales: 50
Unique Stores in Store Master: 50

Unique Customers in Sales: 5000
Unique Customers in Customer Master: 5000

```
sales_sku_missing = ~sales["sku_id"].isin(skus["sku_id"])
sales_store_missing = ~sales["store_id"].isin(stores["store_id"])
sales_customer_missing = ~sales["customer_id"].isin(customers["cust_id"])

print("Sales SKUs not in SKU Master:", sales_sku_missing.sum())
print("Sales Stores not in Store Master:", sales_store_missing.sum())
print("Sales Customers not in Customer Master:", sales_customer_missing.sum())
```

Sales SKUs not in SKU Master: 0
Sales Stores not in Store Master: 0
Sales Customers not in Customer Master: 159821

```
~sales["customer_id"].isin(customers["cust_id"])
```

	customer_id
0	False
1	True
2	True
3	False
4	False
...	...
641838	False
641839	False
641840	False
641841	False
641842	False

641843 rows × 1 columns

dtype: bool

Start coding or [generate](#) with AI.

data cleaning

```
# Convert date columns to datetime

sales["date"] = pd.to_datetime(sales["date"])
promotions["start_date"] = pd.to_datetime(promotions["start_date"])
promotions["end_date"] = pd.to_datetime(promotions["end_date"])

inventory["last_restock_date"] = pd.to_datetime(inventory["last_restock_date"])
inventory["snapshot_date"] = pd.to_datetime(inventory["snapshot_date"])

stores["opening_date"] = pd.to_datetime(stores["opening_date"])
customers["registration_date"] = pd.to_datetime(customers["registration_date"])

print("Date conversion completed successfully.")
```

Date conversion completed successfully.

```
print(sales.dtypes)
```

date	datetime64[ns]
store_id	int64
sku_id	int64
customer_id	float64
quantity	int64
unit_price	float64
total_value	float64
channel	object

```
discount_pct      float64
dtype: object
```

logical date consistency check

```
# 1. Check promotion dates
invalid_promotions = promotions[
    promotions["start_date"] > promotions["end_date"]
]

print("Invalid promotion date records:", len(invalid_promotions))

# 2. Check inventory dates
invalid_inventory = inventory[
    inventory["last_restock_date"] > inventory["snapshot_date"]
]

print("Invalid inventory date records:", len(invalid_inventory))

# 3. Check store opening dates against sales
store_dates = sales.merge(
    stores[["store_id", "opening_date"]],
    on="store_id",
    how="left"
)

invalid_store_dates = store_dates[
    store_dates["date"] < store_dates["opening_date"]
]

print("Sales before store opening:", len(invalid_store_dates))

# 4. Check customer registration dates against sales
customer_dates = sales.merge(
    customers[["cust_id", "registration_date"]],
    left_on="customer_id",
    right_on="cust_id",
    how="left"
)

invalid_customer_dates = customer_dates[
    customer_dates["customer_id"].notna() &
    (customer_dates["date"] < customer_dates["registration_date"])
]

print("Sales before customer registration:", len(invalid_customer_dates))
```

```
Invalid promotion date records: 0
Invalid inventory date records: 0
Sales before store opening: 0
Sales before customer registration: 241985
```

```
total_sales = len(sales)

print("Total sales records:", total_sales)
print("Sales before customer registration:", len(invalid_customer_dates))

percentage = (len(invalid_customer_dates) / total_sales) * 100

print(f"Percentage: {percentage:.2f}%")
```

```
Total sales records: 641843
Sales before customer registration: 241985
Percentage: 37.70%
```

```
display(
    invalid_customer_dates[
        ["date", "customer_id", "cust_id", "registration_date"]
    ].head(20)
)
```

	date	customer_id	cust_id	registration_date
0	2021-01-01	2961.0	2961.0	2024-12-19
3	2021-01-01	3694.0	3694.0	2025-03-16
4	2021-01-01	1252.0	1252.0	2024-11-20
5	2021-01-01	1286.0	1286.0	2024-04-02
7	2021-01-01	3160.0	3160.0	2023-12-06
8	2021-01-01	3870.0	3870.0	2021-01-14
9	2021-01-01	1033.0	1033.0	2021-09-23
10	2021-01-01	4161.0	4161.0	2025-02-28
11	2021-01-01	4914.0	4914.0	2024-04-11
12	2021-01-01	826.0	826.0	2022-03-29
16	2021-01-01	3399.0	3399.0	2022-10-04
17	2021-01-01	347.0	347.0	2022-11-14
19	2021-01-01	3680.0	3680.0	2023-06-20
20	2021-01-01	740.0	740.0	2024-02-09
21	2021-01-01	1998.0	1998.0	2024-04-27
22	2021-01-01	1319.0	1319.0	2021-12-27
24	2021-01-01	3632.0	3632.0	2022-04-20
26	2021-01-01	4170.0	4170.0	2025-05-22
27	2021-01-01	664.0	664.0	2024-02-05
29	2021-01-01	751.0	751.0	2023-08-06

Start coding or [generate](#) with AI.